OpenPaths, formalised in Lean 4

The model-gateway API, and a machine-checked proof that
every request receives its best available execution

A report generated from the Lean sources of this repository

Abstract

This document reports on a Lean 4 formalisation of the OpenPaths model-gateway API — an OpenAI-compatible endpoint that routes an inference request across many models and providers — together with the proof that the gateway performs the *best available execution* of every request it accepts. The core is small and completely explicit: a model catalogue, a request with its constraints, an admission check, a policy-determined lexicographic ranking, and an endpoint that authenticates, routes over the models that are both admissible and healthy, and bills the caller. The best-execution theorem, `OpenPaths.handle_optimal`, states that no admissible healthy catalogue entry ranks better than the model the request was actually run on; it specialises, per routing policy, to genuinely minimal price, minimal latency, maximal benchmark score, or minimal blended price-versus-quality objective. Around that core the repository builds nine further layers — a source-level mirror of the published implementation and a bridge proving the two agree, self-hosting, an autonomous control loop, fleet economics, a batch query server, a transport layer, a complexity lattice, a holographic self-audit, a general architecture module, and zero-knowledge proofs of the port and of skill circuits. Every statement quoted here is proved in the repository: the whole project builds with no `sorry`, no `axiom` declarations of its own, and an axiom closure consisting only of Lean's three standard axioms.

Contents

1 Introduction

1.1 The object being formalised

A *model gateway* is an HTTP service that speaks a single, OpenAI-compatible protocol and forwards each request to one of many upstream inference providers. The client sends a prompt, some constraints (a budget, a deadline, a modality, perhaps a pinned model), and a routing preference; the gateway picks a model, runs the request on it, and bills the caller. Two questions immediately arise, and neither is answered by testing:

1. **Is the execution legal?** Does the model that ran the request actually satisfy every constraint the client stated — the budget, the latency ceiling, the modality, the context window, the pinned id, the provider allow-list?
2. **Is the execution the best one?** Of all the models that could legally have served this request and were reachable at the time, is the one that did serve it optimal for the client's stated preference?

This project answers both by proof. The gateway is written as a total, computable function in Lean 4, and the two questions become theorems about that function, checked by the Lean kernel. Because the definitions are computable, the same definitions the theorems are about also *run*: a worked catalogue is decided by kernel evaluation, and a compiled binary executes the identical code path.

1.2 What "best execution" means here

Best execution is stated relative to three things that the client controls and one that the gateway observes:

- the *admission check* `eligible req m` — a decidable predicate collecting every constraint the request carries;
- the *ranking* `rankKey req m` — a lexicographic triple of natural numbers, smaller being better, whose leading coordinate is chosen by the request's routing policy;
- the *catalogue* — the list of models the gateway lists at all;
- the *health* predicate — which of those models are reachable right now.

The headline theorem is then, informally:

> If an authenticated call returns an execution e, then for every catalogue entry m that is eligible for the request and healthy, $\mathrm{rankKey}(req, e.\mathrm{model}) \leq \mathrm{rankKey}(req, m)$.

This is a statement of optimality against the *actual* alternatives, not against a fixed table: it quantifies over the catalogue and the health state, whatever they are. Sections 4 and 5 give the formal statement and the proof; Section 6 instantiates it on a worked catalogue where the optimum, the bill and the fallback behaviour are all decided by computation.

1.3 How to read this document

Every Lean fragment displayed in this document is extracted mechanically from the sources in this repository by `paper/extract.py`, so the figures are copies of the code that is compiled and checked, doc comments included. Statements are quoted verbatim; proofs are quoted where they are short and are otherwise described in prose with the lemma names that carry them, so that any claim in the prose can be located in the sources. The names used throughout are fully qualified within the `OpenPaths` namespace unless stated otherwise.

2 Verification status

The repository is a single Lean 4 library built against Mathlib. The project contains no `axiom` declarations and no `sorry`; the axiom closure of its theorems consists of Lean's three standard axioms, `propext`, `Quot.sound` and `Classical.choice`, which is the ordinary closure of any development using classical logic and quotients.

Module group	Files	Lines	Theorems	Subject
`OpenPaths/` (core)	15	1 789	98	API, router, execution, catalogue, bridge
`OpenPaths/Mirror/`	7	1 644	79	source-level model of the implementation
`OpenPaths/SelfHost/`	4	859	57	the system submitted to itself
`OpenPaths/Autonomy/`	7	1 712	100	planner, trace geometry, commitments, loop
`OpenPaths/Fleet/`	9	1 869	132	rent-versus-buy economics
`OpenPaths/BatchQuery/`	9	1 935	110	batch query server, payment, arbitration
`OpenPaths/Transport/`	6	1 518	76	transports, integrity, best route
`OpenPaths/Complexity/`	6	1 420	117	the capacity lattice
`OpenPaths/Holographic/`	9	3 735	144	self-audit, views, circuits
`OpenPaths/Architecture/`	6	1 259	62	packaging, towers, defect calculus
`OpenPaths/Zkp/`	4	926	56	zero-knowledge proof of the port
`ZkPoP/`	11	2 178	79	zero-knowledge proofs of skill circuits
Total (including roots)	96	21 263	1 112	

Table 1: Size of the development, counted from the sources. "Theorems" counts `theorem` and `lemma` declarations.

The counts above are line- and declaration-level measurements of the current sources. The repository additionally carries an introspection harness (`introspection/`) that re-derives such numbers from the built environment and from build logs, and records them in `STATUS.md`.

3 The API model

3.1 Endpoints, providers and modalities

Three finite enumerations fix the surface: `Modality` (chat, image, video, music, speech, embedding, transcription), `Provider` (the named upstreams plus an open-ended `other` constructor), and `Endpoint` (the OpenAI-compatible routes). An endpoint determines the modality it serves, and distinct endpoints serve distinct modalities:

```
def Endpoint.modality : Endpoint → Modality
  | .chatCompletions => .chat
  | .imagesGenerations => .image
  ...

theorem Endpoint.modality_injective : Function.Injective Endpoint.modality
```

Injectivity is not decoration: it is used downstream (in the complexity module) to collapse a dispatch pair onto a single node, and it is what makes "the execution serves the endpoint the request arrived at" a consequence of "the execution serves the requested modality".

3.2 Models and requests

A model is a record of the metadata the router needs. Money is measured in integer nano-dollars and time in integer milliseconds, so every quantity the gateway compares is a natural number and every comparison is decidable — which is what makes the worked examples checkable by the kernel.

```
/-- A routable model together with the metadata the router needs.

* `pricePerInputToken` / `pricePerOutputToken` are in nano-dollars per token;
* `latencyMs` is the expected end-to-end latency in milliseconds;
* `quality` is a benchmark score on the scale `0 … qualityScale`;
* `available` records whether the gateway currently lists the model at all.
-/
structure Model where
  id : String
  provider : Provider
  modality : Modality
  supportsTools : Bool
  supportsStreaming : Bool
  contextWindow : ℕ
  pricePerInputToken : ℕ
  pricePerOutputToken : ℕ
  latencyMs : ℕ
  quality : ℕ
  available : Bool
  deriving DecidableEq, Repr, Inhabited
```

A request carries the modality and token counts, the capability requirements, and the optional constraints.

```
/-- A request submitted to the gateway, together with its routing constraints.

`pinnedModel` models the OpenAI-compatible `model` field when the client names a
specific model; leaving it `none` is the gateway's auto-routing mode. -/
structure Request where
  modality : Modality
  promptTokens : ℕ
  maxOutputTokens : ℕ
  needsTools : Bool
  needsStreaming : Bool
  /-- If `some id`, only the model with that id may serve the request. -/
  pinnedModel : Option String
  /-- If `some ps`, only providers in `ps` may serve the request. -/
  allowedProviders : Option (List Provider)
  /-- If `some b`, the execution must cost at most `b` nano-dollars. -/
  maxCost : Option ℕ
  /-- If `some t`, the chosen model's expected latency must be at most `t` ms. -/
  maxLatencyMs : Option ℕ
  minQuality : ℕ
  policy : RoutePolicy
  /-- Price (nano-dollars) the client is willing to pay per point of quality,
  used by `RoutePolicy.balanced`. -/
  qualityWeight : ℕ
  deriving DecidableEq, Repr, Inhabited
```

3.3 Price and objective

```
/-- The price, in nano-dollars, of serving `req` on `m`, billed on the prompt and on
the maximum number of output tokens the client allows. -/
def cost (req : Request) (m : Model) : ℕ :=
  m.pricePerInputToken * req.promptTokens + m.pricePerOutputToken * req.maxOutputTokens
```

Billing is on the prompt and on the *maximum* number of output tokens the client allows, which is the conservative reading: the bill the gateway proves bounds is the worst case the client authorised. The `balanced` policy scalarises price against quality at a rate the client states:

```
/-- Scalarised objective used by `RoutePolicy.balanced`. -/
```

```
def balancedScore (req : Request) (m : Model) : ℕ :=
  cost req m + req.qualityWeight * qualityDeficit m
```

Here `qualityDeficit m` is `qualityScale - m.quality`, with `qualityScale` fixed at 100, and `Model.WellFormed m` says that the score lies on that scale. Truncated natural subtraction makes `qualityDeficit` total; well-formedness is exactly the hypothesis needed to turn a statement about deficits back into a statement about scores, and it is carried explicitly wherever that step is taken.

3.4 Admission

The admission check is one boolean-valued function collecting every clause. Nothing is implicit: a model that fails any clause is not a candidate, and no later stage of the gateway may resurrect it.

```
/-- **Eligibility**: `m` may serve `req`.  This is the complete admission check the
gateway performs before ranking candidates. -/
def eligible (req : Request) (m : Model) : Bool :=
  m.available &&
  (m.modality == req.modality) &&
  (!req.needsTools || m.supportsTools) &&
  (!req.needsStreaming || m.supportsStreaming) &&
  decide (tokenFootprint req ≤ m.contextWindow) &&
  decide (req.minQuality ≤ m.quality) &&
  withinBound (cost req m) req.maxCost &&
  withinBound m.latencyMs req.maxLatencyMs &&
  m.matchesPin req.pinnedModel &&
  m.providerAllowed req.allowedProviders
```

Each clause has a corresponding extraction lemma — `eligible_available`, `eligible_modality`, `eligible_tools`, `eligible_streaming`, `eligible_contextWindow`, `eligible_minQuality`, `eligible_budget`, `eligible_latency`, `eligible_pin`, `eligible_provider` — so that any property of an admitted model can be recovered from the single fact that it was admitted. These are the lemmas through which every client-facing legality theorem later factors.

4 The router

4.1 Ranking

The router ranks admissible candidates by a lexicographic triple of naturals, smaller being better. The routing policy decides which coordinate dominates; the remaining two coordinates are deterministic tie-breakers, so the ranking is total and the router's answer does not depend on unspecified behaviour.

```
/-- The ranking key: a lexicographic triple of natural numbers, smaller is better.
The policy decides which coordinate dominates; the remaining coordinates act as
deterministic tie-breakers. -/
def rankKey (req : Request) (m : Model) : ℕ ×ₗ ℕ ×ₗ ℕ :=
  match req.policy with
  | .cheapest => toLex (cost req m, toLex (m.latencyMs, qualityDeficit m))
  | .fastest => toLex (m.latencyMs, toLex (cost req m, qualityDeficit m))
  | .highestQuality => toLex (qualityDeficit m, toLex (cost req m, m.latencyMs))
  | .balanced => toLex (balancedScore req m, toLex (m.latencyMs, qualityDeficit m))
```

4.2 Specification versus implementation

`rankKey` lives in Mathlib's lexicographic product $\mathbb{N} \times_{\text{l}} \mathbb{N} \times_{\text{l}} \mathbb{N}$. That is the right *specification*: the optimality theorems mean what they mean because $\leq$ there is genuine lexicographic order, with

all of Mathlib's order theory available. It is a poor *implementation*: deciding $<$ on a doubly nested `Lex` product runs through the order-instance tower on every comparison, and driving a selection over such a key through the Lean interpreter can exhaust the interpreter stack.

The development therefore separates the two and proves them equal. The same three coordinates are packaged as a plain triple, compared by arithmetic alone:

```
/-- The three coordinates of `rankKey`, as an ordinary triple of naturals. -/
def rankTriple (req : Request) (m : Model) : ℕ × ℕ × ℕ :=
  match req.policy with
  | .cheapest => (cost req m, m.latencyMs, qualityDeficit m)
  | .fastest => (m.latencyMs, cost req m, qualityDeficit m)
  | .highestQuality => (qualityDeficit m, cost req m, m.latencyMs)
  | .balanced => (balancedScore req m, m.latencyMs, qualityDeficit m)
```

```
/-- Strict lexicographic comparison of triples of naturals, by arithmetic alone. -/
def tripleLt (a b : ℕ × ℕ × ℕ) : Bool :=
  decide (a.1 < b.1) ||
    (decide (a.1 = b.1) &&
      (decide (a.2.1 < b.2.1) || (decide (a.2.1 = b.2.1) && decide (a.2.2 < b.2.2))))
```

```
/-- The router's runtime comparison: is `m₁` strictly better than `m₂` for `req`? -/
def rankLt (req : Request) (m₁ m₂ : Model) : Bool :=
  tripleLt (rankTriple req m₁) (rankTriple req m₂)
```

with `tripleLt_iff` proving that the boolean comparison decides the lexicographic order, and `rankLt_iff` transporting that to `rankKey`:

$$\text{rankLt } \mathit{req}\ m_1\ m_2 = \text{true} \iff \text{rankKey } \mathit{req}\ m_1 < \text{rankKey } \mathit{req}\ m_2.$$

4.3 Selection

Selection is a left fold that keeps the best candidate seen so far, over the sublist of catalogue entries satisfying a predicate:

```
/-- Choose a `rankKey`-minimal model among the catalog entries satisfying `p`.
Ties are broken by catalog order, so the choice is deterministic.

This is `List.argmin (rankKey req)` --- see `bestOf_eq_argmin` --- written out as a fold
over `rankLt`, so that evaluating it never touches the lexicographic order instances. -/
def bestOf (req : Request) (p : Model → Bool) (catalog : List Model) : Option Model :=
  (catalog.filter p).foldl
    (fun best m =>
      match best with
      | none => some m
      | some b => if rankLt req m b then some m else some b)
    none
```

and the fold is proved to be exactly the lexicographic `argmin` it replaces:

```
/-- The implementation is the specification: `bestOf` is `List.argmin` of `rankKey`. -/
theorem bestOf_eq_argmin {req : Request} {p : Model → Bool} {catalog : List Model} :
    bestOf req p catalog = (catalog.filter p).argmin (rankKey req) := by
```

Proof sketch. Unfold `List.argmin` to its own fold over `List.argAux` and prove the two folds agree for *every* accumulator, by induction on the list. In the cons case the two step functions are compared on `none` (both return the head) and on `some b`, where a case split on whether `rankKey req a < rankKey req b` reduces both sides to the same branch — using `rankLt_iff` in one direction and its contrapositive in the other. This is the only place where the arithmetic implementation meets the order-theoretic specification; everything downstream is stated about `rankKey`. □

The router is selection under the admission check:

```
/-- The router: the best eligible model for `req` in `catalog`, if any. -/
def route (req : Request) (catalog : List Model) : Option Model :=
  bestOf req (eligible req) catalog
```

Three facts about `bestOf` carry the whole development: the selected entry is a member of the catalogue (`bestOf_mem_catalog`), it satisfies the predicate it was selected under (`bestOf_pred`), and it is minimal among catalogue entries satisfying that predicate:

```
/-- Minimality: nothing satisfying `p` in the catalog beats the selected model. -/
theorem bestOf_minimal {req : Request} {p : Model → Bool} {catalog : List Model}
    {m m' : Model} (h : bestOf req p catalog = some m)
    (hmem : m' ∈ catalog) (hp : p m' = true) :
    rankKey req m ≤ rankKey req m' :=
  List.le_of_mem_argmin (List.mem_filter.2 ⟨hmem, hp⟩) (bestOf_eq_argmin ▶ h)
```

Together with `bestOf_eq_none_iff` and `bestOf_isSome_iff` — the selection is empty exactly when nothing passes the filter — these five lemmas are the entire interface. Note that they are proved once, for an arbitrary predicate `p`; the router instantiates `p` with eligibility and the endpoint instantiates it with eligibility *and* health, so the health-aware endpoint inherits optimality rather than re-deriving it.

4.4 Optimal routing

```
/-- **Optimality.** No eligible model in the catalog has a better rank than the routed
one. -/
theorem route_optimal {req : Request} {catalog : List Model} {m m' : Model}
    (h : route req catalog = some m) (hmem : m' ∈ catalog) (hel : eligible req m' = true) :
    rankKey req m ≤ rankKey req m' :=
  bestOf_minimal h hmem hel
```

Alongside it, the router is total exactly when it should be: `route_eq_none_iff` says it declines precisely when no catalogue entry is eligible, and `route_isSome_iff` is its positive form. Legality is read off the admission check through the extraction lemmas of Section 3: `route_within_budget`, `route_within_latency`, `route_respects_pin`, `route_modality` and `route_fits_context`.

4.5 Policy-specific optimality

`route_optimal` is a statement about a lexicographic key. What a client wants to hear is a statement about money, or milliseconds, or quality. The bridge is the projection of a lexicographic inequality onto its leading coordinate:

```
/-- Reading off the dominant coordinate of a lexicographic comparison. -/
theorem lex_fst_le {a b : ℕ} {x y : ℕ ×ₗ ℕ}
    (h : (toLex (a, x) : ℕ ×ₗ ℕ ×ₗ ℕ) ≤ toLex (b, y)) : a ≤ b := by
  rcases Prod.Lex.toLex_le_toLex.1 h with h' | ⟨h', -⟩
  · exact le_of_lt h'
  · exact le_of_eq h'
```

With it, each policy yields its own optimality theorem; the `cheapest` case is representative:

```
/-- Under `RoutePolicy.cheapest` the router really does return a cheapest eligible
execution. -/
theorem route_cheapest_cost_le {req : Request} {catalog : List Model} {m m' : Model}
    (hpol : req.policy = .cheapest) (h : route req catalog = some m)
    (hmem : m' ∈ catalog) (hel : eligible req m' = true) :
    cost req m ≤ cost req m' := by
  have := route_optimal h hmem hel
  rw [rankKey, rankKey, hpol] at this
  exact lex_fst_le this
```

The other three are `route_fastest_latency_le` (minimal latency), `route_highestQuality_quality_ge` (maximal benchmark score, under well-formedness of both scores, where truncated subtraction is undone by `omega`) and `route_balanced_score_le` (minimal blended objective).

4.6 Structural properties

Two theorems say that the answer is a property of the *set* of offers rather than of the accidents of how the catalogue was written down or assembled.

```
/-- The quality of the answer does not depend on the order in which the catalog is
listed: reordering can change which of two equally-ranked models is picked, but never
the rank achieved. -/
theorem route_rank_perm_invariant {req : Request} {c₁ c₂ : List Model} {m₁ m₂ : Model}
    (hperm : c₁.Perm c₂) (h₁ : route req c₁ = some m₁) (h₂ : route req c₂ = some m₂) :
    rankKey req m₁ = rankKey req m₂ :=
  le_antisymm
    (route_optimal h₁ (hperm.mem_iff.2 (route_mem_catalog h₂)) (route_eligible h₂))
    (route_optimal h₂ (hperm.mem_iff.1 (route_mem_catalog h₁)) (route_eligible h₁))
```

Permuting the catalogue can change *which* of two equally ranked models is returned — ties are broken by catalogue order, deliberately, so that the gateway is deterministic — but never the rank achieved. The proof is antisymmetry applied to `route_optimal` in both directions.

```
/-- Growing the catalog never hurts: any request that could be served before can still
be served, at least as well. -/
theorem route_mono_of_subset {req : Request} {c c' : List Model} {m : Model}
    (hsub : ∀ x ∈ c, x ∈ c') (h : route req c = some m) :
    ∃ m', route req c' = some m' ∧ rankKey req m' ≤ rankKey req m := by
```

Growing the catalogue never hurts: any request that could be served before can still be served, at least as well. Finally `route_filter_eligible` states that pre-filtering the catalogue to the eligible models does not change the decision, which is the correctness statement for the obvious optimisation.

5 Best execution, end to end

5.1 The endpoint

The endpoint adds two things to the router: authentication, and health. Health is a predicate on models — whatever the gateway currently believes about reachability — and the health-aware router selects under the conjunction of admission and health:

```
/-- The best model that is both eligible and healthy. -/
def routeHealthy (health : Health) (req : Request) (catalog : List Model) : Option Model :=
  bestOf req (fun m => eligible req m && health m) catalog
```

The responses are an execution or a typed error:

```
/-- Errors the gateway can return. -/
inductive ApiError
  /-- No `Authorization` header / API key was supplied. -/
  | missingApiKey
  /-- The client pinned a model id the gateway does not know. -/
  | modelNotFound
  /-- Nothing in the catalog satisfies the request's constraints. -/
  | noEligibleModel
  /-- Models exist for the request, but every one of them is currently down. -/
  | allProvidersDown
  deriving DecidableEq, Repr, Inhabited
```

```
/-- A completed execution: which model ran, and what the caller is billed. -/
structure Execution where
  model : Model
  billedCost : ℕ
  latencyMs : ℕ
  deriving DecidableEq, Repr, Inhabited
```

and the endpoint itself is:

```
/-- The endpoint, for an authenticated caller. -/
def handleAuthed (health : Health) (catalog : List Model) (req : Request) :
    Except ApiError Execution :=
  match routeHealthy health req catalog with
  | some m => .ok ⟨m, cost req m, m.latencyMs⟩
  | none =>
    if pinMissing req catalog then .error .modelNotFound
    else if (route req catalog).isSome then .error .allProvidersDown
    else .error .noEligibleModel
```

```
/-- The gateway endpoint. -/
def handle (apiKey : Option String) (health : Health) (catalog : List Model)
    (req : Request) : Except ApiError Execution :=
  match apiKey with
  | none => .error .missingApiKey
  | some _ => handleAuthed health catalog req
```

Note the order of the failure branches. A pinned model the gateway does not list is `modelNotFound` — never a silent substitution. Otherwise, if the *unconstrained* router would have found something, the failure is an outage (`allProvidersDown`); only if nothing was eligible at all is it `noEligibleModel`. Each of these three readings is later proved to be justified.

5.2 When the gateway succeeds

```
/-- The endpoint succeeds exactly when some eligible model is up. -/
theorem handle_isOk_iff {key : String} {health : Health} {catalog : List Model}
    {req : Request} :
    (∃ e, handle (some key) health catalog req = .ok e) ↔
      ∃ m ∈ catalog, eligible req m = true ∧ health m = true := by
```

Both directions matter. Left to right, a success certifies that a suitable model existed and was up. Right to left — the liveness half — the gateway cannot decline while a usable model exists: if some catalogue entry is eligible and healthy then the selection is non-empty, and the endpoint returns its execution.

5.3 The execution is legal

A successful response is accompanied by a full set of legality theorems, each proved by pushing the corresponding extraction lemma through `handle_ok_eligible`: the executed model is in the catalogue (`handle_ok_mem_catalog`), is eligible (`handle_ok_eligible`), was healthy (`handle_ok_healthy`), the caller is billed exactly its price (`handle_ok_billed`), the reported latency is its latency (`handle_ok_latency`), the bill is within budget (`handle_ok_within_budget`), the latency within the ceiling (`handle_ok_within_latency`), a pin is honoured exactly (`handle_ok_respects_pin`), the modality is served (`handle_ok_modality`), the prompt plus the reserved completion fit in the context window (`handle_ok_fits_context`), and a request arriving at a given endpoint is executed on a model serving that endpoint's modality (`handle_ok_serves_endpoint`).

5.4 The best-execution theorem

```
/-- **Best execution.** No eligible, healthy catalog entry ranks better than the model
the gateway actually ran the request on. -/
theorem handle_optimal {key : String} {health : Health} {catalog : List Model}
    {req : Request} {e : Execution} {m : Model}
    (h : handle (some key) health catalog req = .ok e)
    (hmem : m ∈ catalog) (hel : eligible req m = true) (hh : health m = true) :
    rankKey req e.model ≤ rankKey req m :=
  bestOf_minimal (handle_ok_routeHealthy h) hmem (by simp [hel, hh])
```

Proof. By `handle_ok_routeHealthy`, a successful response was produced by the health-aware selection: `routeHealthy health req catalog = some e.model`. The competitor m lies in the catalogue and satisfies both conjuncts of the predicate under which that selection was made. Minimality of the selection (`bestOf_minimal`) gives exactly the required inequality. □

As with the router, the lexicographic statement specialises per policy through `lex_fst_le`; the `cheapest` case reads:

```
/-- Under the `cheapest` policy the caller is billed the minimum possible amount. -/
theorem handle_cheapest_billed_le {key : String} {health : Health} {catalog : List Model}
    {req : Request} {e : Execution} {m : Model} (hpol : req.policy = .cheapest)
    (h : handle (some key) health catalog req = .ok e)
    (hmem : m ∈ catalog) (hel : eligible req m = true) (hh : health m = true) :
    e.billedCost ≤ cost req m := by
  have hle := handle_optimal h hmem hel hh
  rw [rankKey, rankKey, hpol] at hle
  rw [handle_ok_billed h]
  exact lex_fst_le hle
```

that is: *the caller is billed no more than the price of any model that could legally have served the request and was up.* The companions are `handle_fastest_latency_le`, `handle_highestQuality_quality_ge` and `handle_balanced_score_le`.

5.5 Fallback

Health-awareness could in principle degrade routing quality. It does not, when it does not have to:

```
/-- If the router's first choice is healthy, the execution is exactly as good as the
unconstrained optimum: health-aware fallback never degrades a healthy route. -/
theorem handle_rank_eq_route_rank_of_healthy {key : String} {health : Health}
    {catalog : List Model} {req : Request} {e : Execution} {m : Model}
    (hroute : route req catalog = some m) (hh : health m = true)
    (h : handle (some key) health catalog req = .ok e) :
    rankKey req e.model = rankKey req m :=
  le_antisymm
    (handle_optimal h (route_mem_catalog hroute) (route_eligible hroute) hh)
    (route_optimal hroute (handle_ok_mem_catalog h) (handle_ok_eligible h))
```

The proof is antisymmetry between two optimality statements: the executed model beats the routed one (it is eligible and, by hypothesis, healthy), and the routed model beats the executed one (it is eligible, and the router optimises over a superset).

When the first choice *is* down, the request is still served:

```
/-- Fallback: even if the first choice is down, the request is still executed --- on a
different model --- as long as some other eligible model is up. -/
theorem handle_falls_back {key : String} {health : Health} {catalog : List Model}
    {req : Request} {m m' : Model}
    (hdown : health m = false)
```

```
    (hmem : m' ∈ catalog) (hel : eligible req m' = true) (hh : health m' = true) :
    ∃ e, handle (some key) health catalog req = .ok e ∧ e.model ≠ m := by
```

The proof is the liveness half of `handle_isOk_iff` plus `handle_ok_healthy`: the execution is healthy, and the downed model is not, so they are different models.

5.6 Every error is justified

The three error branches are not merely reachable; each is a claim about the world, and each claim is proved.

```
/-- If the gateway reports `allProvidersDown`, then indeed some model could have served
the request but none of them was up. -/
theorem handle_allProvidersDown {key : String} {health : Health} {catalog : List Model}
    {req : Request} (h : handle (some key) health catalog req = .error .allProvidersDown) :
    (∃ m ∈ catalog, eligible req m = true) ∧
      ∀ m ∈ catalog, eligible req m = true → health m = false := by
```

Similarly `handle_noEligibleModel` proves that nothing in the catalogue could have served the request, and `handle_modelNotFound` proves that the pinned id really is absent from the catalogue. A gateway that returned `allProvidersDown` when in fact nothing was eligible, or `noEligibleModel` during an outage, would fail these theorems.

5.7 The shape of the argument

The whole best-execution result rests on a short chain, which is worth stating plainly because it is what makes the development robust to changes in the ranking or the admission check:

1. **Implementation = specification.** `bestOf_eq_argmin`: the arithmetic fold the gateway runs is the lexicographic `argmin`.
2. **Selection is minimal.** `bestOf_minimal`, for an arbitrary predicate.
3. **The endpoint's predicate is admission ∧ health.** So the executed model is minimal among exactly the models that could have served the request.
4. **Projection.** `lex_fst_le`: a lexicographic inequality bounds its leading coordinate, which the policy has arranged to be the quantity the client cares about.
5. **Extraction.** The admission check is decomposed once, and every legality theorem is an instance.

Changing the ranking changes step 4 only; adding a clause to the admission check adds one extraction lemma and leaves steps 1–4 untouched.

6 The worked catalogue

`Catalog.lean` fixes a thirteen-entry catalogue and a 30 000-token streaming chat request with tool calling and a 2 000-token completion budget, then decides, by kernel evaluation, which execution the gateway performs under each policy and constraint set. The prices, latencies and benchmark scores are illustrative stand-ins chosen to exercise the router; what is verified is the routing logic, not a vendor price list.

Under the `cheapest` policy the router selects `deepseek-chat` and the endpoint bills 10 300 000 nano-dollars, i.e. $0.0103:

```
/-- With the `cheapest` policy the router picks the cheapest eligible model. -/
theorem route_cheapest : route baseRequest catalog = some deepseekChat := by decide
```

Model	Provider	Context	In	Out	Latency	Quality
`gpt-5-mini`	openai	400 000	250	2 000	900	82
`gpt-5`	openai	400 000	1 250	10 000	2 400	95
`claude-haiku`	anthropic	200 000	800	4 000	700	80
`claude-sonnet`	anthropic	200 000	3 000	15 000	2 000	93
`gemini-flash`	google	1 000 000	300	2 500	600	84
`gemini-pro`	google	1 000 000	1 250	10 000	2 600	94
`llama-3.3-70b`	groq	131 072	590	790	200	72
`grok-4`	xai	256 000	3 000	15 000	2 500	92
`deepseek-chat`	deepseek	128 000	270	1 100	3 000	86
`mistral-large`	mistral	128 000	2 000	6 000	1 500	85
`gpt-4-legacy`	openai	128 000	30 000	60 000	3 000	70
`tiny-ctx`	mistral	8 192	10	20	100	60
`flux-pro`	other	4 096	0	40 000 000	6 000	88

Table 2: The worked catalogue. Prices are nano-dollars per token. `gpt-4-legacy` is listed but marked unavailable, `tiny-ctx` exercises the context-window check and `flux-pro` is an image model, exercising the modality check.

```
/-- The end-to-end execution of the request, with its bill (10.3 million
nano-dollars, i.e. 0.0103 USD). -/
theorem execute_cheapest :
    handle (some "sk-openpaths-demo") allHealthy catalog baseRequest =
      .ok ⟨deepseekChat, 10300000, 3000⟩ := by decide
```

Both are closed by `decide`: the kernel evaluates the gateway on this catalogue and checks the result. The corresponding optimality claim is *not* decided — it is obtained from the general theorem, instantiated here:

```
/-- ... and that really is the minimum price over all eligible models. -/
theorem cheapest_is_minimal :
    ∀ m ∈ catalog, eligible baseRequest m = true →
      cost baseRequest deepseekChat ≤ cost baseRequest m :=
  fun _ hm hel => route_cheapest_cost_le rfl route_cheapest hm hel
```

The remaining scenarios in the file, each checked the same way: quality-first routing selects `gpt-5` (and `quality_is_maximal` certifies it); latency-first selects the Groq-hosted Llama at 200 ms; adding a 800 ms ceiling *and* a quality floor of 80 moves the answer to `gemini-flash`; the `balanced` policy at 6 000 000 nano-dollars per quality point selects `gpt-5`; restricting to Anthropic selects `claude-haiku`; pinning `grok-4` serves exactly that model; pinning an unlisted id fails with `modelNotFound`; a budget of 100 nano-dollars fails with `noEligibleModel`; a missing key fails with `missingApiKey`; a DeepSeek outage falls back to `gpt-5-mini` at 11 500 000 nano-dollars rather than failing; and a total outage reports `allProvidersDown`.

```
/-- The router's first choice is unavailable, so the request is executed on the next
best model instead of failing. -/
theorem execute_cheapest_with_deepseek_down :
    handle (some "sk-openpaths-demo") deepseekDown catalog baseRequest =
      .ok ⟨gptMini, 11500000, 900⟩ := by decide
```

The same computations run from the compiled executable (`lake exe openpaths all`), so the numbers in the theorems and the numbers the binary prints are the same numbers.

7 The source-level mirror, and the bridge

The core of Sections 3–5 models the gateway's *documented behaviour*. The modules under `OpenPaths/Mirror/` model the published *implementation* instead, following its source structure:

the health tracker with per-key cool-downs and linear backoff; model configurations, aliases and temporary provider routes with resolution and retries; candidate ordering with its strategy normalisation, its blended input-plus-output rate and its sentinel for unpriced models; the autorouter with its `auto-*` names, task-tier hints and cosine similarity; and the pricing function with cache-hit clamping, long-context rates, truncation and a minimum charge.

What is proved there is that resolution only ever offers usable routes (a registered provider, a healthy key), that the chain is non-empty on success, that the configured primary heads it when usable, and that no provider/model pair repeats; that the autorouter's suggestion is preferred but never fatal; that ordering is a permutation of the resolved chain, sorted, and stable; and that the executed candidate is the head of the ordered chain.

`Bridge.lean.tex` then connects the two. A resolved candidate is abstracted to a behaviour-level `Model` whose per-million-token rates are read as the price of one million prompt tokens and one million completion tokens, and a distinguished request asks for exactly that, so the abstract `cost` of an abstracted candidate *is* the implementation's blended rate. The implementation's registry-and-cooldown check is abstracted to a behaviour-level health predicate. The adequacy statements are then:

- every abstracted candidate is admitted by the blended request (`eligible_toModel`), and a priced candidate's blended rate is exactly its abstract cost (`blendedPrice_eq_cost`);
- under a price strategy, the candidate the source-level gateway executes is a cheapest model of the abstracted chain (`bestExecution_cost_le`);
- the behaviour-level router run over that chain bills exactly what the implementation bills (`bestExecution_cost_eq_route_cost`), the abstract endpoint succeeds whenever the implementation does (`handle_abs_isOk`) and bills the same amount (`bestExecution_cost_eq_handle_billed`), and the two decline together (`route_abs_eq_none_iff`).

In other words, the optimality theorem proved about the abstract gateway is a theorem about the modelled implementation as well, rather than about a convenient idealisation of it. The hypotheses are discharged concretely on a four-entry slice of the worked configuration, where both layers execute the same model and bill the same 420 000 000 nano-dollars, checked by evaluation.

8 The layers built on top

The remaining modules apply the same discipline — define the object, state what the client is entitled to, prove it — to the questions that surround a gateway in production. This section summarises each; the repository's per-module documents give the full statement lists.

8.1 Self-hosting: the system as its own request

`OpenPaths/SelfHost/` submits the gateway's own sources to the gateway, as a request to rewrite them into a Lean formalisation. The rewrite is proved total and convergent: one pass makes every module Lean, preserves module names one-for-one, and a corpus is a fixed point of the rewrite exactly when it is already fully formal, so iteration stabilises after the first pass. The job then receives its best execution: it runs whenever some capable model is up, on a model that is listed, healthy, tool-capable, above the quality floor and large enough to hold the whole system together with its formalisation; no eligible healthy model ranks better (`selfHost_optimal`), and a provider outage does not stop the run. The bootstrap converges: a successful run returns a fully formal system covering exactly the modules it was given, and running the system on that output returns the same output.

8.2 Autonomy: planning, geometry, commitments, control

`OpenPaths/Autonomy/` closes a loop around the gateway. A deterministic GOAP planner returns a goal-reaching plan of minimal search cost within a horizon, and declines exactly when the task is unsolvable within it; tuning its weights never changes *whether* a task is solvable, only which plan is found. Execution traces are turned into a coordinate system generated by the runs themselves, in which the true cost of a run is a linear functional of its feature vector. The covariance of the resulting point cloud is symmetric and positive semidefinite, explained variance plus reconstruction error along a unit direction equals the total, and a principal coordinate is literally the spread of a per-execution statistic. The control loop admits proposals from an untrusted advisor through a deterministic gate; against *any* advisor, adversarial ones included, the final score plus the number of accepted moves is at most the initial score, so the score never rises and only finitely many moves are ever accepted, and replaying the logged moves reproduces the final parameters exactly. Users steer the loop by committing to a secret policy polynomial: an honest opening always verifies, a dishonest one passes only on a set of setup points bounded by the degrees involved, and away from that set an accepted move installs exactly the user's own value. Finally, the planner *is* the router: serving a request is a one-move planning problem whose optimal plan costs exactly what the `cheapest` router bills.

8.3 Fleet: renting instead of buying

`OpenPaths/Fleet/` answers a whole queue by renting compute rather than buying inference per token, and then decides between the two. The searches are exhaustive and their optima are proved: no hour of the price curve and no contiguous window is quoted lower than the one chosen, with preemption risk priced in; the greedy packer is a legal batching of the queue using the fewest possible batches (an exchange argument), and the saving is exactly one provisioning charge per batch avoided; pooling a queue under a volume tier never costs more than buying in lots; offloading layers to local hardware never costs more when a local token-layer is not dearer than a rented one, with the latency counterweight stated alongside; and the plan search returns a cheapest feasible plan over hour, machine and local/remote split.

```
/-- **Optimality of the plan**: no feasible plan --- no other hour, machine or
local/remote split --- serves the workload for less. -/
theorem bestPlan_le {dep : Deployment} {wl : Workload} {p q : Plan}
    (h : bestPlan dep wl = some p) (hq : PlanFeasible dep wl q) :
    planCost dep wl p ≤ planCost dep wl q :=
  bestBy_le h (mem_candidatePlans_iff.2 hq)
```

The buy-or-rent decision is proved to take an option costing no more than any other legal option, and the self-improving planner — which observes the market rather than knowing it — is proved never to regress: learned quotes only move downwards, a replan never increases the planned bill, and the planned bill is non-increasing along a whole run of market ticks.

8.4 The batch query server

`OpenPaths/BatchQuery/` is a second server: a content-addressed store of public data, batched inference requests over it, ranking, payment, reputation, stake and arbitration. The integrity property that makes untrusted peers usable is proved with no assumption on the hash and no trust in the peer: whatever a fetch returns hashes to the name that was asked for. Batching is a permutation of the queue, one batch per distinct block, and cheaper than serving requests one at a time. Ranking admits an answer only on checkable grounds — it answers that query, from the named block, inside the deadline, above the quality floor — and the winner beats every other admissible answer:

```
/-- **Best execution.**  The winner scores at least as high as every admissible answer
to the query. -/
```

```
theorem winner_max {w : QoS} {q : Query} {rs : List Response} {r r' : Response}
    (hr : winner w q rs = some r) (hr' : r' ∈ rs) (had : Admissible q r') :
    score w q r' ≤ score w q r := by
```

Money is conserved by every ledger operation, so slashed stake moves to the treasury rather than vanishing; a provider is paid exactly the bounties of the queries it won; and, with stake at least the bounty, collecting a bounty and then being evicted for it leaves a cheat no better off than never having served. Because the data is public and content-addressed, disputes are decidable by recomputation: an honest provider is never found against, whoever challenges, and a wrong answer always is when the data is available.

8.5 Transports: best execution for bytes

`OpenPaths/Transport/` serves blocks — the node's own sources included — over whatever transports are configured: store-and-forward, peer-to-peer, content-addressed, cold storage, public mirrors, trackers, repository hosts, plain HTTPS, and an extension point. Transports are untrusted: a plugin hands back bytes, the server re-hashes them, and anything that does not match the name asked for is discarded. Integrity holds with no assumption on the transport at all. Choosing between transports is the same best-execution problem the gateway solves for models, with money and latency in place of price and quality:

```
/-- **Best execution.**  No transport that could have delivered the block on the
caller's terms would have done so at a lower total cost than the route taken. -/
theorem best_optimal {h : Hash} {pol : Policy} {r : Registry} {req : Request} {q : Quote}
    (hq : r.best h pol req = some q) :
    ∀ q' ∈ r.quotes h req, q.score pol ≤ q'.score pol :=
  bestBy_le hq
```

so nothing on offer would have been cheaper under the node's policy, the caller's budget and deadline are honoured, one surviving route is enough, registering more plugins never makes execution worse, and publishing never loses a copy. The node's own disk is modelled as one more plugin — free and instantaneous, registered first — so "what you host yourself is served off your own disk" is a consequence of optimality rather than a special case, and it survives a total blackout of every other transport.

8.6 The complexity lattice

`OpenPaths/Complexity/` measures how much data an object of this formalisation can hold, comparatively: an object is bracketed between two reference structures of exactly known capacity, and the bracket is narrowed by constraints until the object is pinned to a node or breaks against one. Fitting a node is proved equivalent to injecting into its underlying set, with the pigeonhole converse for breaks, so a capacity bound is mathematics rather than numerology. Reference data entered by hand is tagged as such and cannot be laundered into proved data by anything downstream, and an overflow may only be recorded together with a decomposition proved to sum to it.

8.7 Holographic self-audit

`OpenPaths/Holographic/` turns the formalisation on itself: the routing core is enumerated as a catalogue of typed rows, committed to, and re-derived from bounded fragments of itself. The commitment hash is concrete and is accompanied by an explicit collision, so collision-freedom is carried as a hypothesis wherever it is needed and never quietly assumed. A circuit language with a fuel-indexed evaluator supports meta-circuits that decide claims about circuits, applied to their own descriptions. A *view* is a fragment plus a commitment plus a declared scale plus a query plus a certificate — all five — and a view that verifies certifies a property of the whole catalogue. Coverage is reported as a vector of five axes rather than one ratio, and the loss of every proof

object at the second level is itself a theorem. A ladder of stations spells out what is asserted and, explicitly, what is not.

8.8 The unified architecture

`OpenPaths/Architecture/` states the general architecture the previous two modules instantiate. A contract is what a component must declare; a packaged realization is an implementation with a digest and a proof; an invariant forced by the contract holds of every realization, so clients may depend on it, while the choice of realization remains contingent — and a component with an *undeclared* side condition hands that condition to its clients rather than removing it. Invariants are constant on orbits and factor through the orbit quotient. The endomorphism/retraction distinction is kept honest by a worked pair: the Frobenius map on polynomials over $\mathbb{Z}/p$ is injective with no section, while on a finite field the same formula is bijective. Towers of brackets have antitone gaps, a zero gap pins the object, and a run breaks upwards or downwards at a named rung with every rung below it still holding.

8.9 Zero-knowledge proof of the port

`OpenPaths/Zkp/` lets the gateway prove to a client that its response was the best execution available *without* revealing the price list it was best against. The gateway publishes the request, the execution, a commitment to its private view and its public key, and proves that it knows a private view with that commitment and a key for that public key under which the gateway returns exactly that execution. Every best-execution theorem is re-derived from the claim alone. The protocol is a Schnorr-style Σ-protocol with completeness, special soundness with an explicit extractor, soundness error $1/q$, and perfect honest-verifier zero knowledge — the honest prover's views are exactly the accepting transcripts, so two provers with different secrets are indistinguishable. Fiat–Shamir turns it into a bundle of one group element and one scalar:

```
/-- **A verified proof of the port.**  An honest gateway publishes a bundle that
verifies, and whose statement is backed by a hidden catalog on which the execution the
client received is the best available one: it is a real catalog entry, admissible for
the request, healthy, billed exactly at its price, and no admissible healthy alternative
ranks better. -/
theorem verified_port_best_execution {H : PortWitness → Holographic.Digest} {g : G}
    {st : PortStatement G} {w : PortWitness} {sk : ZMod q}
    (hrel : PortRel H g st w sk) (chal : Challenge q G) (r : ZMod q) :
    (prove g chal st sk r).Verifies g chal st ∧
      H w = st.catalogCommit ∧
      st.execution.model ∈ w.catalog ∧
      OpenPaths.eligible st.request st.execution.model = true ∧
      w.health st.execution.model = true ∧
      st.execution.billedCost = cost st.request st.execution.model ∧
      ∀ m ∈ w.catalog, OpenPaths.eligible st.request m = true → w.health m = true →
        rankKey st.request st.execution.model ≤ rankKey st.request m :=
```

A proof is rejected for every other claim, and any two proofs reusing a commitment reveal the key. The two cryptographic assumptions — hardness of the discrete logarithm, and preimage resistance of the commitment — are stated as a trust boundary, and no theorem depends on either.

8.10 Zero-knowledge proofs of skill circuits

`ZkPoP/` formalises the arithmetization pipeline that turns a typed skill into a circuit: an object language of types and terms with a checker and an evaluator, a polynomial-system layer with a Tseitin-style flattening proved sound and complete, a gadget catalogue (zero-test, multiplexer, Horner evaluation, root membership, Merkle paths), Pedersen commitments with perfect hiding and binding reduced to discrete logarithm, and a three-move protocol with an extractor. Its

non-negotiable bridge is the theorem that makes a proof *about the checker* a statement *about the result*:

```
/-- **The semantic bridge (`checker_sound`).**  If the checker accepts `t` at type `T`
in a context matched by the environment, then evaluation succeeds and returns a value
of exactly type `T`.  Integration of a zkPoP skill is gated on this theorem: it is what
makes a proof *about the checker* a statement *about the result*. -/
theorem checker_sound {ρ : List Val} {Γ : List Ty} (hρ : EnvOk ρ Γ) :
    ∀ (t : Tm) (T : Ty), infer Γ t = some T → ∃ v, eval ρ t = some v ∧ v.ty = T := by
```

Type-tag uniqueness is genuinely load-bearing here — the conditional gate can compare only tags, and it is injectivity of the tag map that upgrades "equal tags" to "equal types" in the satisfiability correspondence — and the wire encoding is proved to be a prefix code, so wire vectors decode uniquely and there is no junk.

9 What is claimed, and what is not

The value of a formalisation depends on being exact about its boundary. The following are proved, in the sense of being checked by the Lean kernel from the definitions given: the router returns a real catalogue entry that passes the admission check; it declines exactly when nothing is eligible; every client constraint is honoured; the entry returned is rank-minimal, and per policy this is minimal price, minimal latency, maximal score or minimal blended objective; the arithmetic implementation of the selection is the lexicographic specification; the ranking achieved is invariant under reordering the catalogue and monotone under enlarging it; the endpoint succeeds exactly when an eligible healthy model exists; the execution is legal, billed at the executed model's price, and optimal among eligible healthy models; fallback never degrades a healthy route and does not fail a request while a usable alternative is up; and each error is justified by the state of the catalogue.

The following are *not* claimed. The prices, latencies and benchmark scores in the worked catalogue are illustrative stand-ins, not a snapshot of a live price list; what is verified is the routing logic, not vendor numbers. The formalisation models a gateway's behaviour and, in the mirror modules, the structure of an implementation — it is not a compilation of that implementation, and no claim is made that deployed binaries refine these definitions beyond what the bridge theorems state. Latency and quality are the advertised figures carried in the catalogue, not measured outcomes: the gateway is proved to optimise the numbers it is given. The cryptographic modules prove protocol-level properties relative to standard hardness assumptions, which are recorded as a trust boundary and are not themselves theorems here. Finally, the reference orders entered by hand in the complexity module are tagged as entered rather than proved, and nothing downstream is permitted to upgrade that status.

10 Reproducing the results

```
lake build                      # the library and all of its proofs
lake build openpaths            # the executable
lake exe openpaths list         # the probes it can run
lake exe openpaths all          # run every probe, one JSON line each
lake exe openpaths run handle/cheapest 1000
```

A successful `lake build` *is* the test result: the worked examples are in-source kernel checks (`decide` and `example` declarations) elaborated by the build itself, so nothing is verified by a separate harness that could disagree with the proofs. The binary executes the same definitions the theorems are about, so `handle/cheapest` prints the execution that `execute_cheapest` proves.

This document is generated from the sources: `paper/extract.py` pulls each displayed declaration out of the Lean files, and `paper/build.sh` compiles `paper/openpaths.tex` to `paper/openpaths.pdf`.

A Index of headline theorems

All names are in the `OpenPaths` namespace (or the sub-namespace of the module named).

Name	What it states
Routing (`Routing.lean`)	
`bestOf_eq_argmin`	the arithmetic fold is the lexicographic `argmin`
`route_mem_catalog`	the answer is a real catalogue entry
`route_eligible`	the answer passes the admission check
`route_optimal`	no eligible model has a better rank
`route_eq_none_iff`	the router declines exactly when nothing is eligible
`route_within_budget`	the budget is never exceeded
`route_within_latency`	the latency ceiling is never exceeded
`route_respects_pin`	a pinned request is served by exactly that model
`route_fits_context`	prompt plus reserved completion fit the context window
`route_cheapest_cost_le`	`cheapest`: minimal price
`route_fastest_latency_le`	`fastest`: minimal latency
`route_highestQuality_quality_ge`	`highestQuality`: maximal score
`route_balanced_score_le`	`balanced`: minimal blended objective
`route_rank_perm_invariant`	the rank achieved is independent of catalogue order
`route_mono_of_subset`	growing the catalogue never hurts
Execution (`Execution.lean`)	
`handle_missing_key`	no API key, no execution
`handle_isOk_iff`	success exactly when an eligible model is up
`handle_optimal`	**best execution**
`handle_cheapest_billed_le`	the bill is minimal among eligible healthy models
`handle_fastest_latency_le`	the latency is minimal
`handle_highestQuality_quality_ge`	the score is maximal
`handle_balanced_score_le`	the blended objective is minimal
`handle_ok_within_budget`	the caller is never billed above budget
`handle_ok_respects_pin`	a pin is honoured exactly
`handle_ok_serves_endpoint`	the endpoint's modality is served
`handle_rank_eq_route_rank_of_healthy`	fallback never degrades a healthy route
`handle_falls_back`	an outage does not fail a servable request
`handle_allProvidersDown`	the outage error is justified
`handle_noEligibleModel`	the no-candidate error is justified
`handle_modelNotFound`	the unknown-pin error is justified
Worked catalogue (`Catalog.lean`)	
`route_cheapest`, `execute_cheapest`	the decided route and its bill
`cheapest_is_minimal`	that bill is the minimum, from the general theorem
`execute_cheapest_with_deepseek_down`	the decided fallback
`handle_badPin`, `handle_impossible`, `handle_outage`	the decided errors
Bridge, and the layers above	
`Bridge.bestExecution_cost_le`	the implementation executes a cheapest abstracted candidate
`Bridge.bestExecution_cost_eq_handle_billed`	both layers bill the same
`SelfHost.selfHost_optimal`	the self-hosting job gets its best execution
`Autonomy.plan_routingTask_cost_eq_route`	the planner's optimum is the router's

Name	What it states
`Fleet.bestPlan_le`	the plan returned is a cheapest feasible plan
`Fleet.bestSourcing_le`	buy-or-rent takes a cheapest legal option
`BatchQuery.winner_max`	the paid answer beats every admissible answer
`BatchQuery.runRound_total`	a round conserves the books
`Transport.best_optimal`	the route taken is the best on offer
`Transport.serve_sound`	never the wrong bytes, whatever the transport
`Zkp.verified_port_best_execution`	the verified bundle backs a best execution
`ZkPoP.Tm.checker_sound`	acceptance by the checker implies a value of that type